UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CONTADURÍA Y ADMINISTRACIÓN

SISTEMA DE UNIVERSIDAD ABIERTA

Alumno: Emanuel Marzano Villalobos

Programación De Dispositivos Móviles

Cristian Cardoso Arellano

Grupo 9697

m6_01

05/11/2026

Enlace a repositorio de la actividad: https://github.com/Mr-X22/android-2027-1/tree/main/M6_01

1. Introducción

En un aeropuerto, la torre de control debe saber en todo momento dónde está cada avión y hacia dónde se dirige, para detectar a tiempo si dos de ellos van a coincidir en el mismo lugar. Esta actividad simula esa situación de forma sencilla: el espacio aéreo se representa como una cuadrícula (grid), cada avión ocupa una casilla y mira hacia una dirección, y en cada paso todos avanzan una casilla.

El objetivo es desarrollar una aplicación Android en Java que muestre esa cuadrícula, permita avanzar y retroceder paso a paso, y detecte las colisiones que ocurren cuando dos o más aviones llegan a la misma casilla al mismo tiempo. Para ello se partió del proyecto base proporcionado por el asesor, que ya contenía la estructura de clases, y se completaron las partes que faltaban.

Antes de programar, el problema se resolvió primero en papel para entender las reglas y llegar a un algoritmo propio, que después se convirtió en código.

2. Objetivo

Realizar un algoritmo para la resolución del problema “Colisión de aviones en un aeropuerto” e implementarlo en una aplicación Android en lenguaje Java que muestre el grid, los aviones con su dirección, las colisiones, el número de pasos y de colisiones, con botones para avanzar al paso siguiente y regresar al paso anterior.

3. Análisis del problema

3.1. Entendiendo las reglas

El aeropuerto es una hoja cuadriculada. Cada casilla se identifica con dos números, como en el juego de batalla naval: la columna (x), que crece hacia la derecha, y la fila (y), que crece hacia abajo. La casilla de arriba a la izquierda es la (0,0).

Cada avión tiene una posición y una dirección. Al avanzar una casilla solo cambia uno de sus números:

- **`Norte (arriba)`**: se le resta 1 a la y.
- **`Sur (abajo)`**: se le suma 1 a la y.
- **`Este (derecha)`**: se le suma 1 a la x.
- **`Oeste (izquierda)`**: se le resta 1 a la x.

Hay colisión cuando dos o más aviones terminan en la misma casilla en el mismo paso. Lo importante es la sincronización: si llegaran a esa casilla en pasos distintos, simplemente pasarían sin tocarse.

3.2. Ejercicio en papel

Para comprobar que entendía las reglas, resolví a mano un caso en un grid de 5×5 con tres aviones:

```
Paso 0:          Paso 1:          Paso 2:
 . . v . .       . . . . .       . . . . .
 . . . . .       . . v . .       . . . . .
> . . . <       . > . < .       . . X . .
 . . . . .       . . . . .       . . . . .
```

El avión A va de (0,2) hacia el este, B de (4,2) hacia el oeste y C de (2,0) hacia el sur. En el paso 1 ninguno comparte casilla, pero en el paso 2 los tres llegan a (2,2). Decidí contar esto como una sola colisión (porque ocurre en un solo lugar) en la que participan tres aviones.

3.3. Alternativa que consideré: predecir los choques

Mi primera idea fue calcular los choques antes de que ocurrieran: ver la casilla inicial de cada avión, contar cuántas casillas hay de distancia con los demás y, como cada uno avanza una casilla por paso, saber en qué paso se encontrarían. Por ejemplo, A y B están en la misma fila a 4 casillas de distancia y se acercan 2 por paso, así que chocan en el paso 4 ÷ 2 = 2.

Esta idea funciona con aviones que van de frente, pero se complica con los que se cruzan en distinta dirección, porque hay que calcular cuándo llega cada uno al cruce. Además, si un avión choca antes con otro, ya no llega a un cruce que el cálculo sí predecía. Como la aplicación de todos modos debe mostrar el grid en cada paso, elegí simular paso por paso: mover a todos una casilla y revisar quién quedó junto.

3.4. Algoritmo de la solución

Mi algoritmo, con mis propias comparaciones

Cada clic en "Paso siguiente" es como un silbatazo: todos los aviones se mueven al mismo tiempo.

1. Para cada avión veo hacia dónde mira y lo muevo una sola casilla (este x + 1, oeste x − 1, sur y + 1, norte y − 1).

2. Ya que todos se movieron, reviso el tablero como en un Sudoku, buscando casillas donde haya más de un avión, igual que se buscan números repetidos.

3. Si encuentro una casilla con 2 o más aviones, coloco ahí una explosión, la cuento como una colisión y esos aviones se eliminan (se van al taller). Los demás siguen volando.

4. Si un avión se sale del grid, sale del radar (se pierde en el Triángulo de las Bermudas) y deja de mostrarse. No cuenta como colisión.

5. Cada paso que calculo lo guardo en la memoria, como una foto del tablero, para no volver a calcularlo y poder regresar a él después.

6. Cuando se presiona "Paso anterior", solo retrocedo a la foto anterior que está en la memoria. Si estoy en el paso 0, no hay foto anterior.

Sobre el punto 4, también pensé en hacer que el avión explotara al chocar con el borde, o en ampliar el mapa. Lo descarté porque el problema trata de colisiones entre aviones (contar el borde aumentaría las colisiones sin que choquen dos aviones) y porque, como los aviones siempre van en línea recta, por grande que sea el mapa tarde o temprano llegan al borde.

4. Desarrollo

4.1. Proyecto y estructura

Se creó el proyecto M6_01 (Empty Views Activity, Java) y se copiaron del proyecto base del asesor las imágenes de los aviones en sus cuatro direcciones y la de la colisión, en la carpeta res/mipmap. Las clases del proyecto base se pasaron una por una, completando las partes marcadas como pendientes. En lugar de abrir directamente el proyecto base, se creó uno nuevo porque el original fue hecho con una versión anterior de Android Studio.

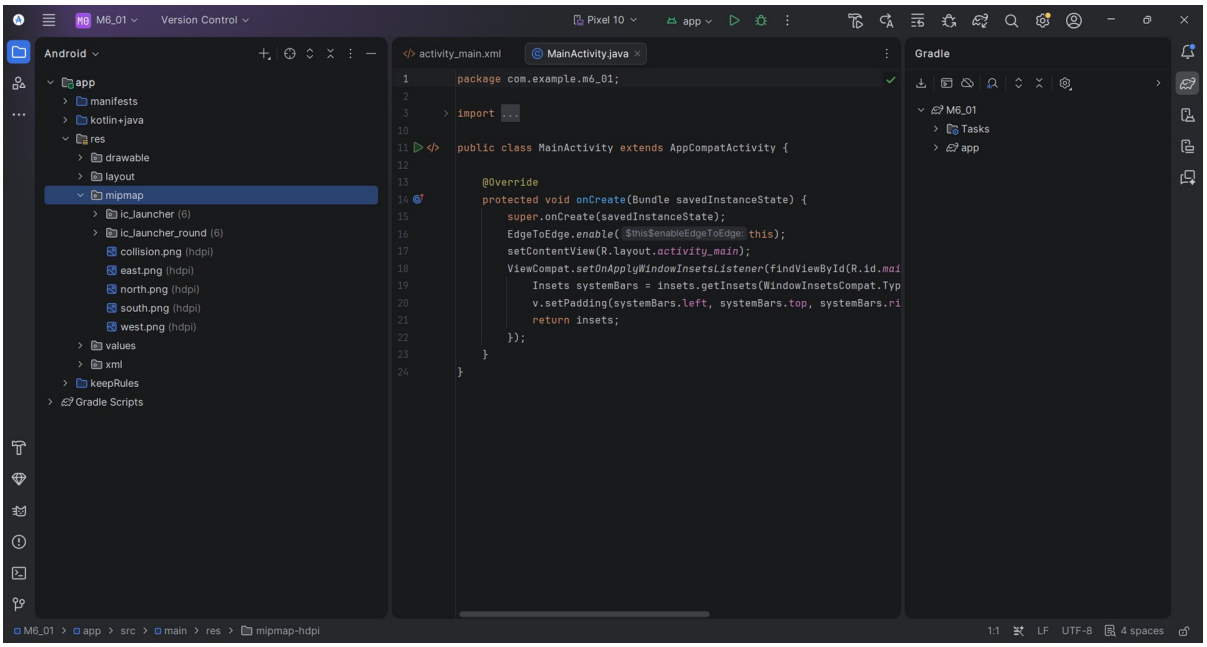


Figura 1. Clases del proyecto e imágenes de los aviones y la colisión en res/mipmap.

Clase	Origen	Qué hace
Direccion	Proyecto base	Las 4 direcciones posibles: NORTH, SOUTH, EAST, WEST.
Avion	Base + completada	Guarda dirección y posición. Se agregaron mover() y copiar().
Colision	Base + completada	Guarda dónde ocurrió un choque. Se agregó su constructor.
Plano	Base + completada	La “foto” de un paso. Se completó prev().
Analizador	Base + completada	Calcula cada paso: mueve, detecta choques, saca a los que salen del radar y guarda la foto.
Aerolineas	Base + caso propio	Casos de prueba: el del asesor y el mío (MI_CASO).
Planificador	Proyecto base	Arma la foto del paso 0 con el caso elegido.
MainActivity	Nueva	Dibuja el grid de 8×8, los contadores y conecta los botones.

4.2. Avion: mover y copiar (punto 1)

A la clase Avion del proyecto base se le agregó el método mover(), que aplica las reglas de movimiento con un switch según la dirección:

```
public void mover() {
    switch (direccion) {
        case NORTH: y = y - 1; break;
        case SOUTH: y = y + 1; break;
        case EAST:  x = x + 1; break;
        case WEST:  x = x - 1; break;
    }
}
```

También se agregó copiar(), que crea un avión nuevo con los mismos datos:

```
public Avion copiar() {
    return new Avion(direccion, x, y);
}
```

Este método resolvió un problema del proyecto base: este movía directamente los mismos aviones del paso anterior, por lo que la “foto” guardada de ese paso también cambiaba y el botón de retroceder no podía funcionar. Con copiar(), cada paso mueve una fotocopia del avión y la foto anterior queda intacta.

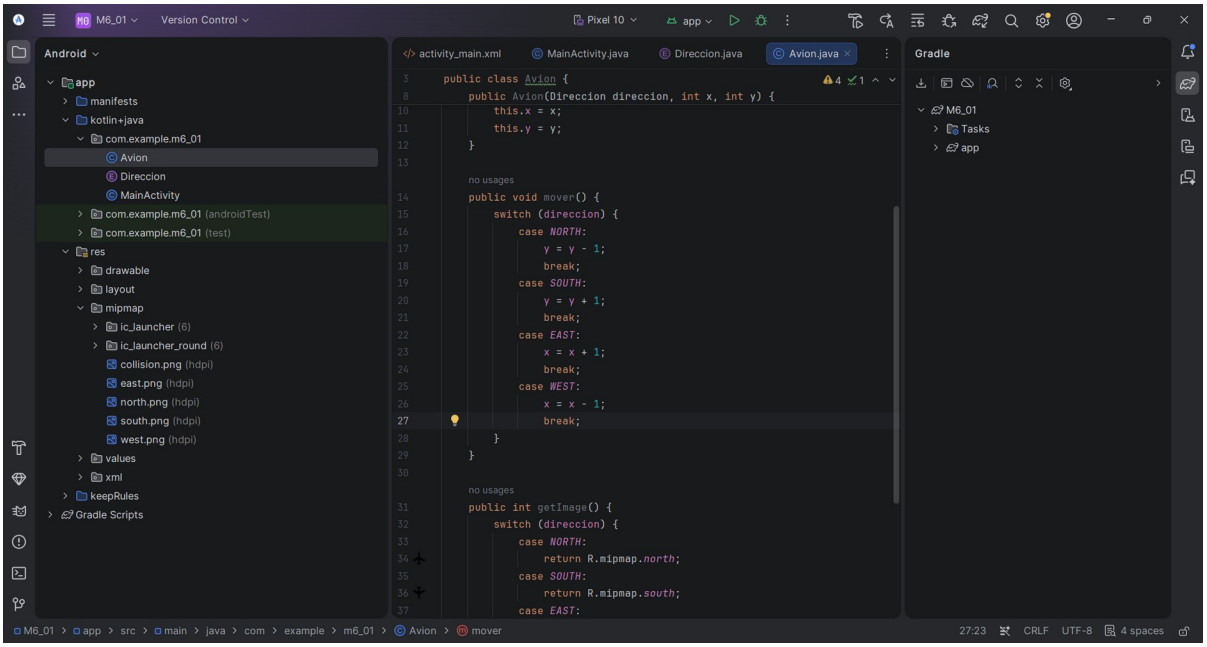


Figura 2. Método mover() en la clase Avion.

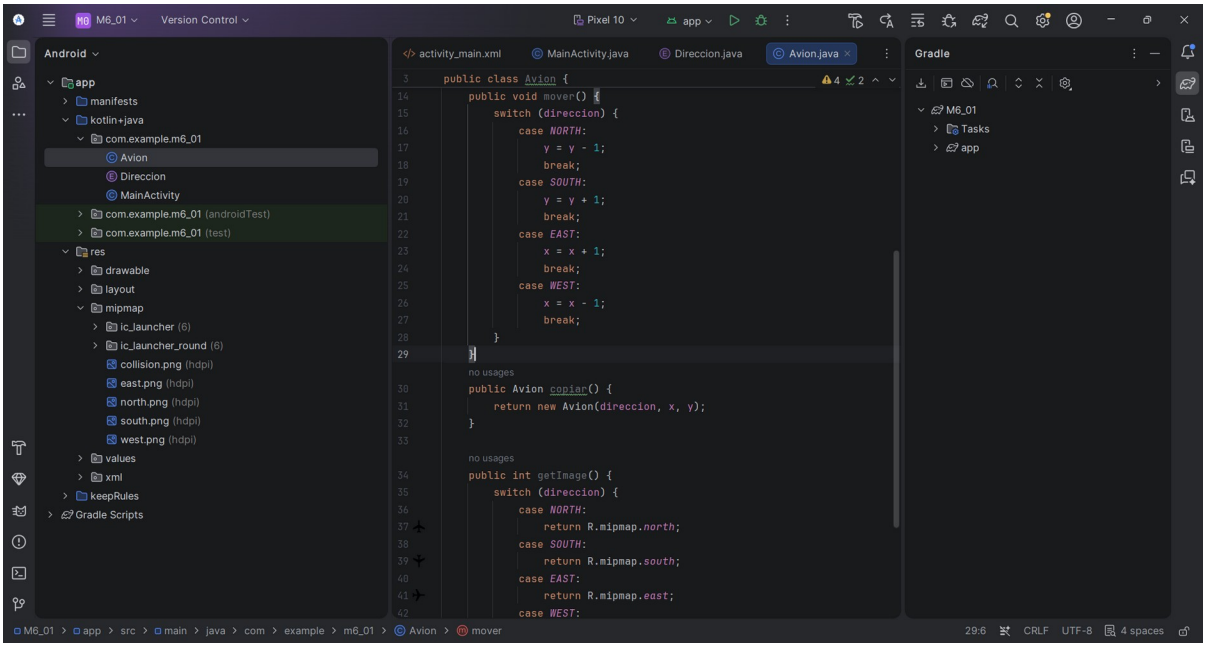


Figura 3. Método copiar() y getImage(), que elige la imagen según la dirección.

4.3. Colision y Plano

En Colision se completó el constructor para guardar la casilla del choque (this.x = x; this.y = y;). Plano representa la foto de un paso: guarda la lista de aviones en vuelo, las colisiones de ese paso, el número de paso y el total de colisiones acumuladas. En él se completó prev() (punto 6):

```
public Plano prev() {
    if (noPaso == 0) {
        return this;
    }
    return Analizador.prev(noPaso - 1);
}
```

Si ya se está en el paso 0 no hay foto anterior y se regresa la misma; si no, se pide a la memoria la foto del paso anterior (noPaso − 1), al contrario de next(), que pide noPaso + 1.

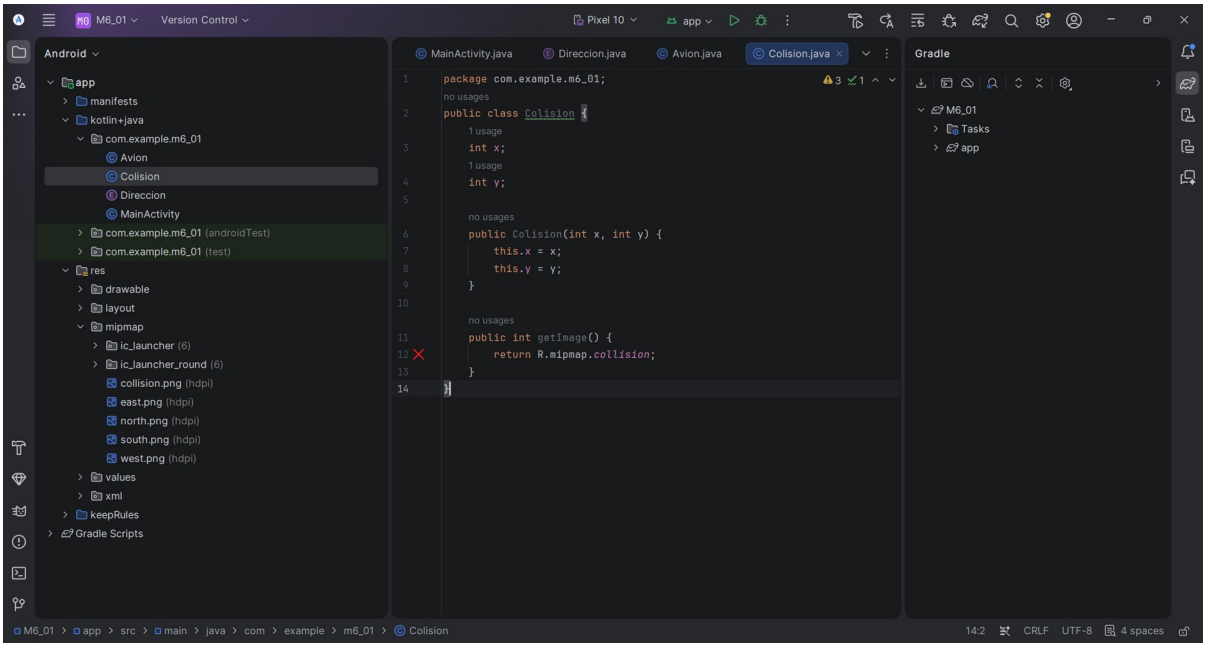


Figura 4. Clase Colision con su constructor.

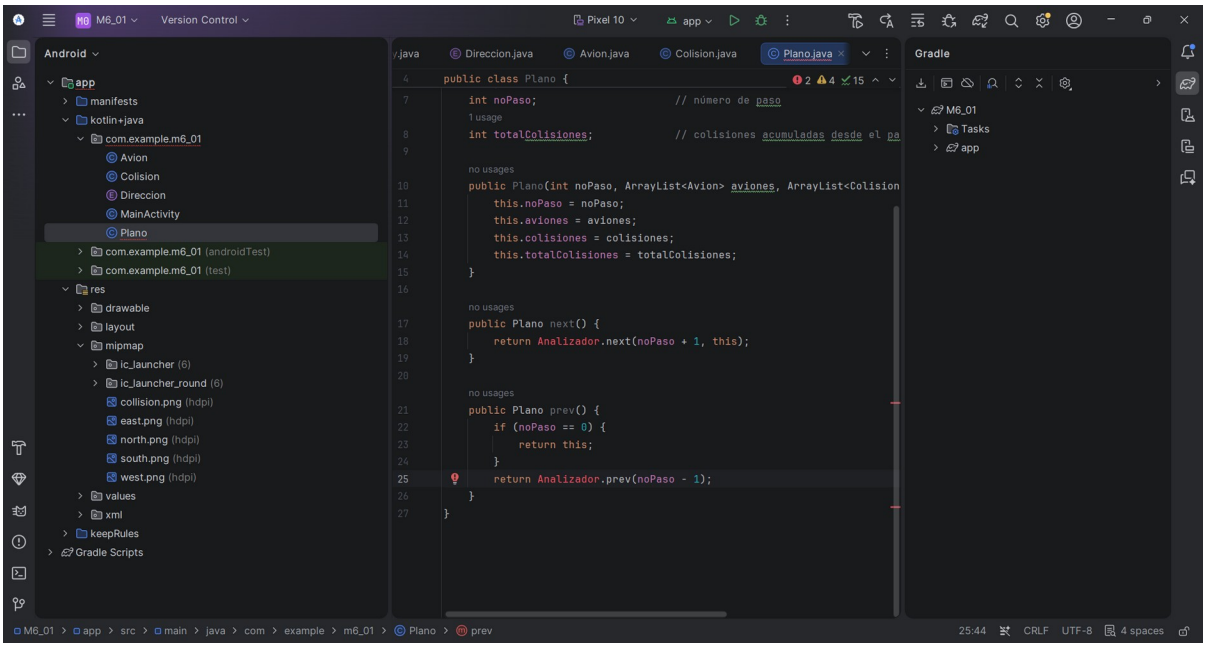


Figura 5. Métodos next() y prev() de la clase Plano.

4.4. Analizador: el cálculo de cada paso (puntos 1 a 5)

El Analizador es donde se aplica el algoritmo. Tiene una memoria (HashMap) que guarda la foto de cada paso usando el número de paso como etiqueta, igual que la lista memo de la actividad del Fibonacci (M5-01). Su método next() hace lo siguiente:

Si la foto ya existe, se toma de la memoria (punto 5)

```
if (memoria.containsKey(noPaso)) {
    return memoria.get(noPaso);
}
```

Mover una copia de cada avión (punto 1)

```
for (Avion avion : plano.aviones) {
    Avion copia = avion.copiar();
    copia.mover();
    movidos.add(copia);
}
```

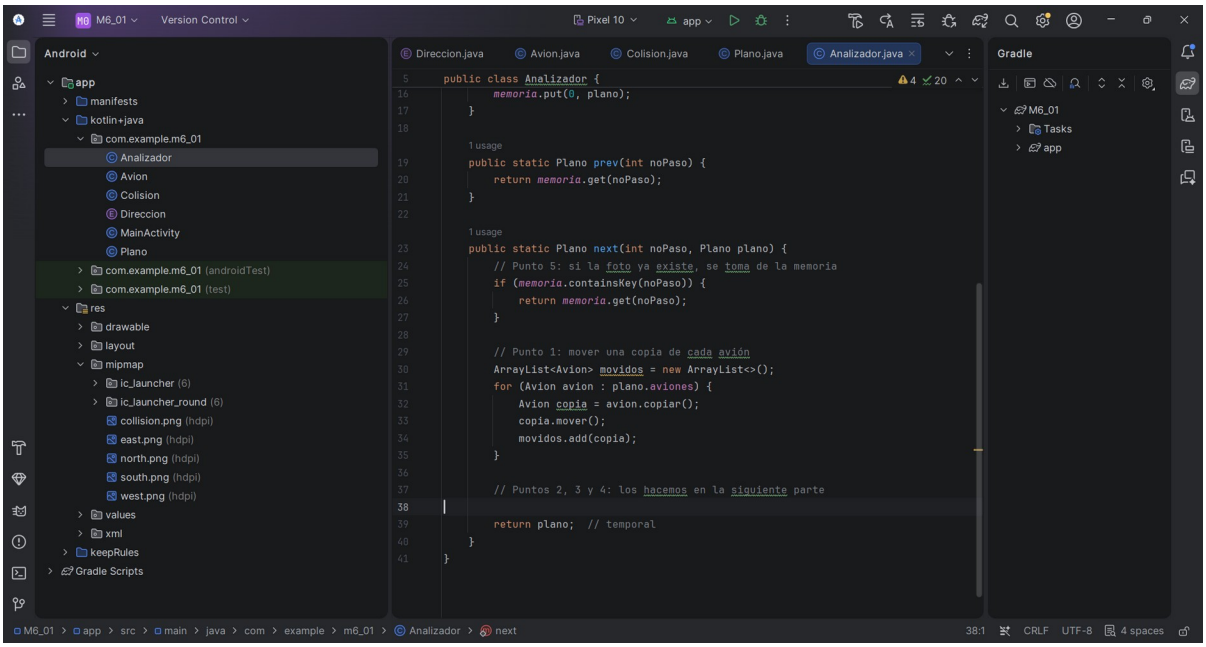


Figura 6. Memoria y movimiento de las copias en el Analizador.

Revisar el tablero como en un Sudoku (puntos 2, 3 y 4)

Para cada avión se cuenta cuántos aviones hay en su misma casilla, comparándolo con todos los demás mediante dos ciclos for, uno dentro de otro:

```
int enLaCasilla = 0;
for (Avion otro : movidos) {
    if (otro.x == avion.x && otro.y == avion.y) {
        enLaCasilla = enLaCasilla + 1;
    }
}
```

El ciclo también cuenta al avión consigo mismo, por lo que un avión solo da 1 y un choque se reconoce cuando enLaCasilla es mayor que 1. Después se decide qué hacer con el avión:

- **`Si chocó`**: se agrega una explosión en esa casilla. El método hayColisionEn() revisa que no exista ya una, para que un choque de tres aviones se dibuje y cuente una sola vez.
- **`Si no chocó y sigue dentro del grid`**: pasa a la lista de aviones del siguiente paso.
- **`Si salió del grid`**: simplemente no se agrega: sale del radar.

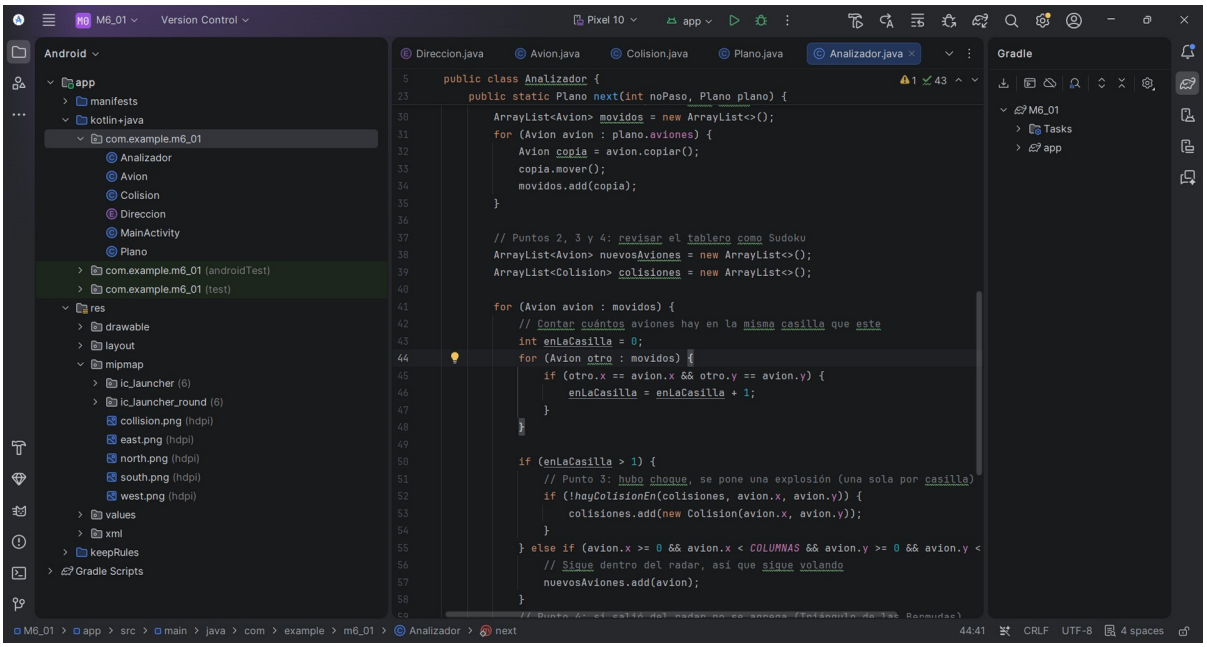


Figura 7. Comparación tipo Sudoku y decisión sobre cada avión.

Guardar la foto (punto 5)

Al final se suma el total de colisiones, se crea la nueva foto y se guarda en la memoria antes de regresarla.

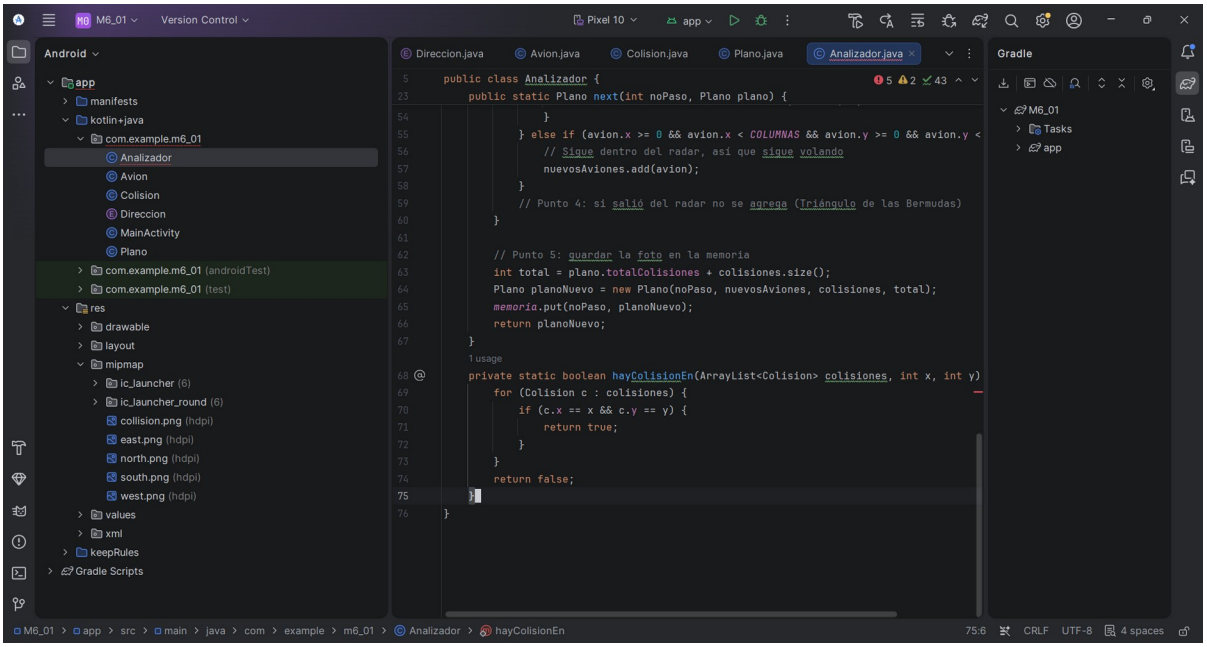


Figura 8. Guardado de la foto en memoria y método hayColisionEn().

4.5. Casos de prueba: Aerolineas y Planificador

En Aerolineas se conservó el caso del asesor (AEROMEXICO, dos aviones de frente) y se agregó MI_CASO con los aviones del ejercicio en papel, más un cuarto avión D para probar que los aviones que no chocan siguen volando y que al salir del grid desaparecen:

```
aviones.add(new Avion(Direccion.EAST, 0, 2));    // A
aviones.add(new Avion(Direccion.WEST, 4, 2));    // B
aviones.add(new Avion(Direccion.SOUTH, 2, 0));   // C
aviones.add(new Avion(Direccion.NORTH, 6, 6));   // D
```

Planificador arma la foto del paso 0 con el caso elegido y la entrega al Analizador. Para probar otro caso basta con cambiar MI_CASO por el nombre del caso; ahí se agregarán los casos de prueba del asesor.

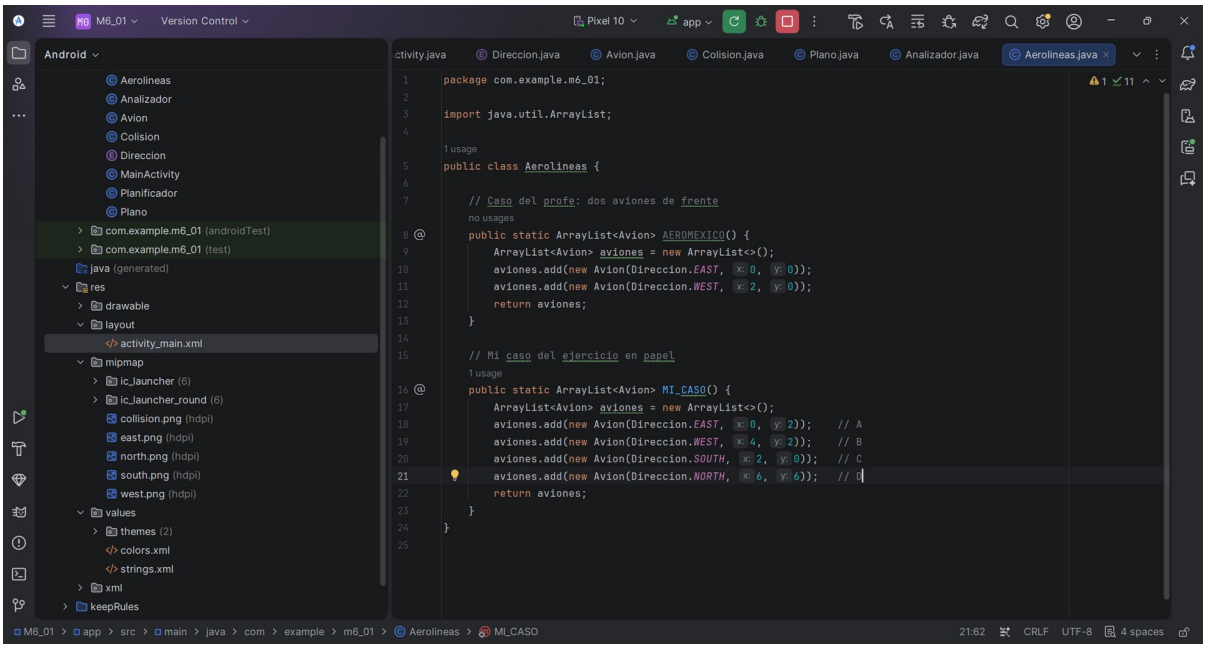


Figura 9. Casos de prueba en la clase Aerolineas.

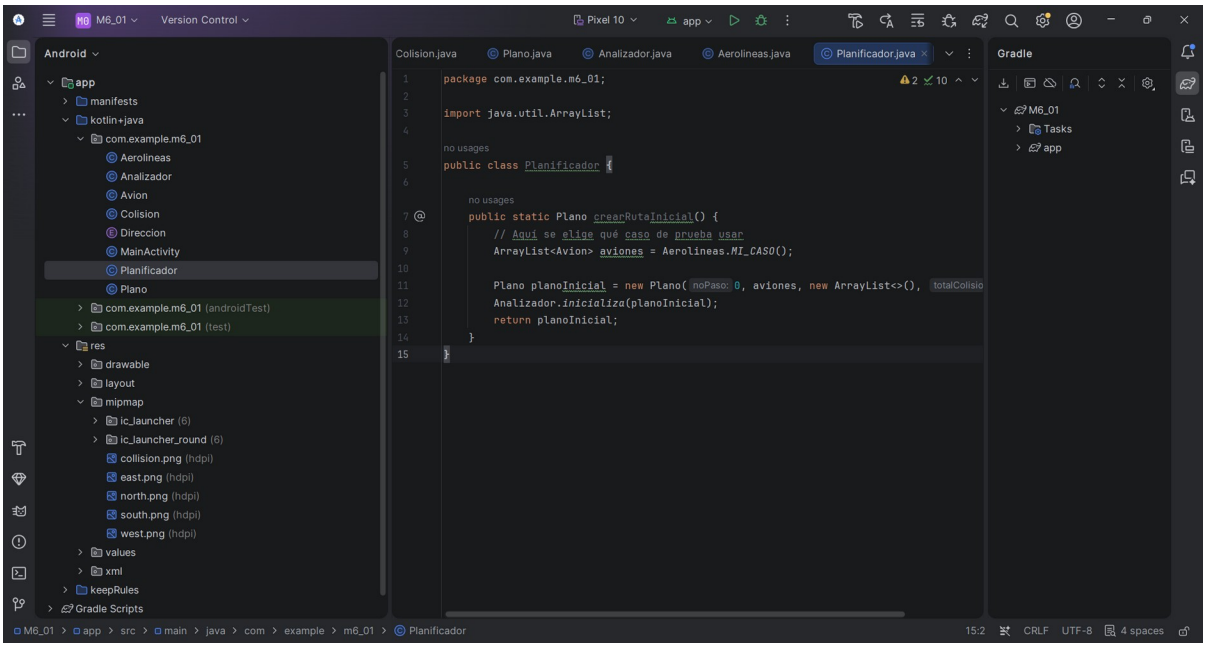


Figura 10. Clase Planificador.

4.6. Pantalla: grid, contadores y botones

El diseño (activity_main.xml) tiene el título, un texto con la información del paso, un GridLayout de 8 columnas y los dos botones; los textos fijos se guardaron en strings.xml. Para que se vean las líneas del grid, el GridLayout tiene fondo gris y cada casilla es blanca con un margen de 1, de modo que el gris se asoma entre ellas como en una hoja cuadriculada.

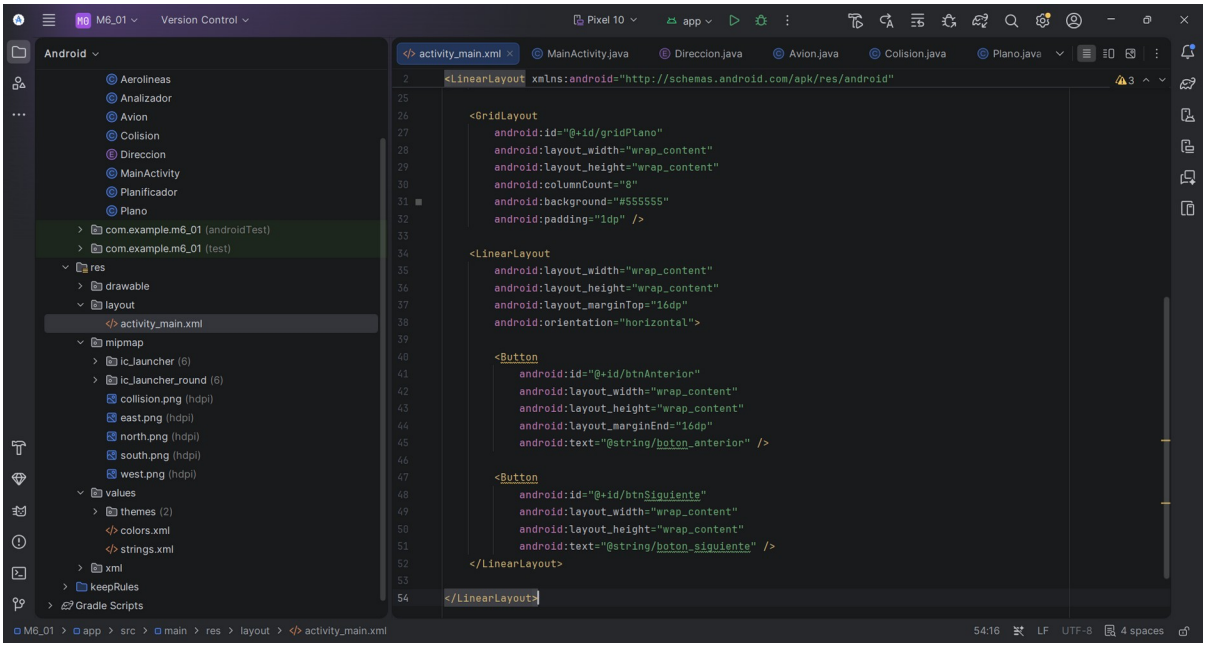


Figura 11. GridLayout y botones en activity_main.xml.

En MainActivity, las 64 casillas se crean desde Java con dos ciclos for (filas y columnas) y se guardan en una tabla celdas[y][x]. El método dibujar() limpia todas las casillas, pinta cada avión con la imagen de su dirección, pinta las explosiones y actualiza los contadores. Las explosiones se dibujan después de los aviones para que queden encima:

```
for (Colision colision : plano.colisiones) {
    celdas[colision.y][colision.x].setImageResource(colision.getImage());
}
```

Los botones cambian la foto con next() o prev() y la vuelven a dibujar.

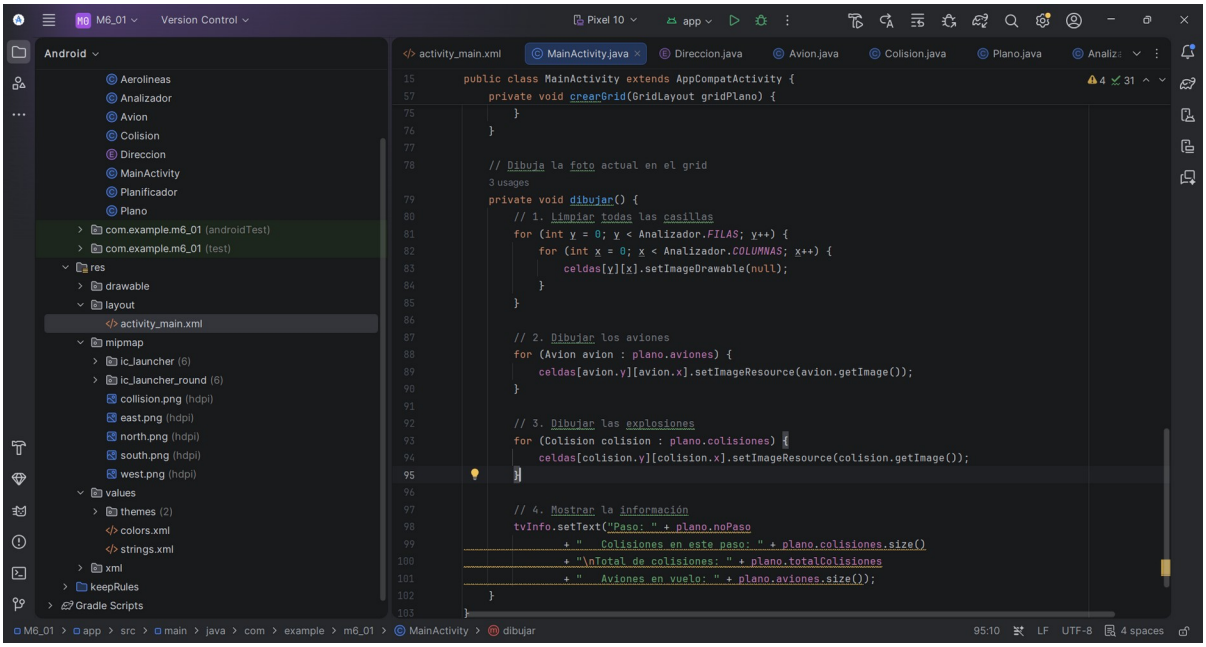


Figura 12. Método dibujar() en MainActivity.

4.7. Pruebas y resultados

La aplicación se ejecutó en el emulador Pixel 10 (Android 17, API 37) con MI_CASO. Antes de correrla se predijo en papel lo que debía pasar en cada paso, y después se comparó con la aplicación:

Paso	Predicción en papel	Resultado en la app	Aviones en vuelo	Colisiones (total)
0	A (0,2), B (4,2), C (2,0), D (6,6)	Igual	4	0
1	Todos avanzan una casilla	Igual	4	0
2	A, B y C chocan en (2,2)	Explosión en (2,2)	1	1
5	D en (6,1)	Igual	1	1
6	D en (6,0), borde superior	Igual	1	1
7	D sale del radar	Grid vacío	0	1

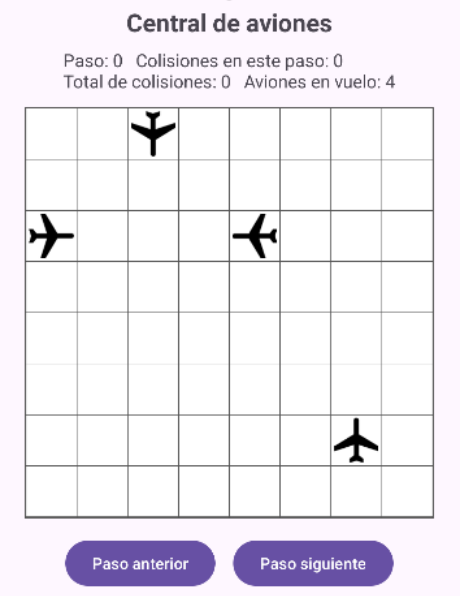


Figura 13. Paso 0: los 4 aviones.

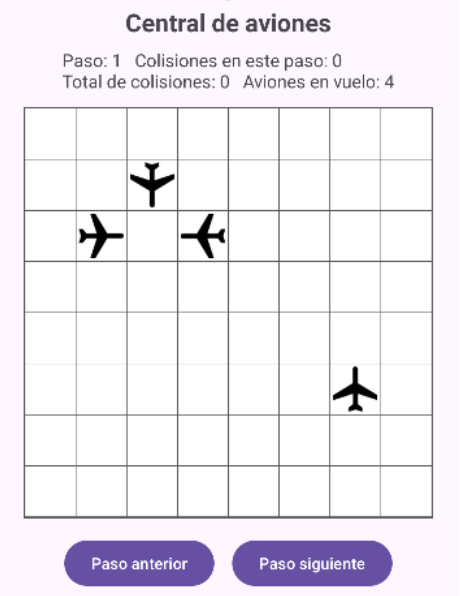


Figura 14. Paso 1: todos avanzan.

En el paso 0 se ven los cuatro aviones con la imagen de su dirección. En el paso 1 cada uno avanzó una casilla y todavía no hay colisiones.



Figura 15. Paso 2: colisión en (2,2).



Figura 16. Paso 5: D sigue volando.

En el paso 2, A, B y C llegan a (2,2): aparece la explosión, se cuenta 1 colisión y queda 1 avión en vuelo, que es D. En el paso 5, D sigue subiendo solo por la columna 6.

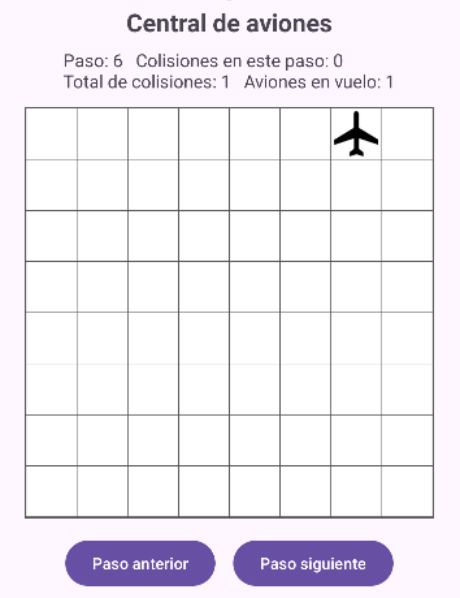


Figura 17. Paso 6 (con "Paso anterior").



Figura 18. Paso 7: D sale del radar.

En el paso 7, D salió por el borde superior: ya no hay aviones en vuelo y el total de colisiones se mantiene en 1, porque salir del grid no cuenta como choque. Desde ahí se presionó "Paso anterior" y la aplicación regresó al paso 6, con D en (6,0), tomando la foto guardada en memoria. Todos los resultados coincidieron con la predicción en papel.

Funcionalidades solicitadas: grid visible, número de pasos, número de colisiones, aviones, dirección del avión, colisión, botón de paso siguiente y botón de paso anterior. Todas se observan en las figuras 13 a 18.

5. Conclusión

Esta fue la actividad más complicada del semestre, porque no bastaba con escribir código: primero había que entender el problema y pensar cómo resolverlo. Resolverlo en papel antes de programar me ayudó mucho, porque al dibujar los pasos me di cuenta de que lo importante es que los aviones lleguen a la misma casilla al mismo tiempo.

Mi primera idea fue predecir los choques contando distancias, pero al analizarla vi que se complicaba con aviones que se cruzan, así que elegí simular paso por paso. Para explicarme el algoritmo usé comparaciones que me resultaron útiles: cada paso es como un silbatazo, la revisión de choques es como buscar repetidos en un Sudoku, los aviones que chocan se van al taller, los que salen del grid se pierden en el Triángulo de las Bermudas y cada paso se guarda como una foto para poder regresar.

También aprendí que un detalle puede romper una función completa: el proyecto base movía los mismos aviones de los pasos anteriores, por lo que retroceder no funcionaba, y se resolvió

moviendo una copia de cada avión. Al final, comprobar que la aplicación hacía exactamente lo que había predicho en papel me dio confianza en que la solución era correcta.

Código fuente de esta actividad: https://github.com/Mr-X22/android-2027-1/tree/main/M6-01

6. Referencias

Cardoso, C. (2024). *Aviones* [Proyecto base]. GitHub. https://github.com/CristianCardosoA/AndroidFCA24-1/tree/main/Aviones

Coding Together ES. (2023, 19 de septiembre). *¿Qué es un HASHMAP en Java y cuáles son sus métodos?* [Video]. YouTube. https://www.youtube.com/watch?v=8g_XrOSe0Qc

González, J. D. M. (2018, 28 de noviembre). *Ciclos anidados*. ProgramarYa. https://www.programarya.com/Cursos/C++/Ciclos/Ciclos-anidados

Google. (s. f.). *GridLayout*. Android Developers. https://developer.android.com/reference/android/widget/GridLayout

Novato Programando. (2024, 15 de mayo). *#16 Arreglos bidimensionales (matrices) | Curso de pseudocódigo desde cero* [Video]. YouTube. https://www.youtube.com/watch?v=wBFVCNS6rZE

Oracle. (s. f.). *HashMap (Java SE 17 & JDK 17)*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/HashMap.html

Recursos de elementos de diseño. (s. f.). Android Developers. https://developer.android.com/guide/topics/resources/drawable-resource?hl=es-419

Simulation algorithms: Types & techniques. (s. f.). StudySmarter. https://www.studysmarter.co.uk/explanations/engineering/automotive-engineering/simulation-algorithms/